

Technology Level: Basic to advanced smartphone/web users

Main Activity: Searching and purchasing products online

Needs: Easy product discovery, reliable cart management, simple checkout, and order tracking

Expected Experience: Fast, secure, mobile-friendly, and easy to use

Seller/Admin Persona

The administrator or store manager manages products, categories, stock quantities, and orders through the administrative functions of the system.

1.2 Core Problem Statement

Traditional purchasing processes can be time-consuming because customers must physically visit stores, compare products manually, and depend on manual order management. Small businesses may also struggle with maintaining product information, inventory, customer orders, and cart data in an organized system.

The proposed e-commerce platform addresses these problems by providing a centralized online shopping system where customers can browse products, search and filter the catalog, manage their carts, and place orders. Administrators can maintain product inventory through controlled CRUD operations.

1.3 Domain Scope

Domain: General Retail E-Commerce

The system focuses on online retail products such as:

Electronics

Clothing and Apparel

Home Accessories

Beauty Products

Books

General Consumer Goods

Out of Scope for Sprint 1

The following advanced features are outside the initial MVP boundary:

Real payment settlement

AI product recommendations

Multi-vendor marketplace management

Advanced analytics

International tax calculation

Complex logistics integration

These features can be considered for future sprints.

2. Minimum Viable Product (MVP) Feature Scope

The MVP contains the core workflows required to operate a basic online shopping platform. The scope is intentionally limited so that it remains feasible within an academic semester.

2.1 MVP Feature Scope Matrix

Category

Feature Name

Description

Priority

Authentication

User Registration & Authentication

User registration, secure password hashing, login, logout, and JWT-based authentication.

High (MVP)

Catalog

Product List & Search

Product browsing with category-based filtering and search functionality.

High (MVP)

Product

Product Details

Display product name, category, price, stock availability, and description.

High (MVP)

Cart

Cart Management

Persistent cart allowing users to add, update, and remove products.

High (MVP)

Checkout

Order Processing

Convert cart items into an order and support a mock or Stripe payment workflow.

High (MVP)

Admin

Inventory Control

Administrative CRUD operations for products, categories, and inventory quantities.

Medium

2.2 Core User Workflow

Customer Workflow

User opens the website.

User registers or logs in.

User browses or searches products.

User filters products by category.

User opens product details.

User adds products to the cart.

User updates or removes cart items if required.

User proceeds to checkout.

System creates an order.

User receives an order confirmation.

Administrator Workflow

Administrator logs in.

Administrator opens the inventory management area.

Administrator adds, edits, or deletes products.

Administrator updates stock quantities.

Administrator manages product categories.

Administrator reviews order information.

3. Tech Stack Selection & Justification

3.1 Frontend Framework — React.js

Selected Technology: React.js

React.js is selected because its component-based architecture supports reusable UI elements such as product cards, navigation bars, search controls, carts, and checkout forms. Compared with developing the interface using plain JavaScript, React provides better component organization and state-management possibilities while remaining suitable for an academic project.

3.2 Backend Infrastructure — Node.js + Express.js

Selected Technology: Node.js with Express.js

Node.js and Express.js provide a lightweight and scalable REST API environment suitable for e-commerce workflows such as authentication, product management, cart operations, and order processing. Express has a large ecosystem and integrates naturally with JavaScript-based frontend development, reducing the complexity of using different programming languages across the application.

3.3 Database Management System — PostgreSQL

Selected Technology: PostgreSQL

PostgreSQL is selected because the application contains strongly related entities such as users, products, categories, orders, and order items. A relational database provides primary keys, foreign keys, constraints, transactions, and data integrity, making it more appropriate for this structured e-commerce model than a document-oriented database such as MongoDB.

3.4 Authentication — JWT

Selected Technology: JSON Web Token (JWT)

JWT can be used for stateless user authentication between the frontend and backend. Passwords will be stored as secure hashes rather than plain text.

3.5 Caching & Asynchronous Processing — Redis (Optional)

Selected Technology: Redis

Redis may be introduced in a later sprint for caching frequently accessed information and supporting session-related or background-processing requirements. It is optional for the initial MVP so that the academic project remains within scope.

3.6 Payment — Mock / Stripe

The MVP will use a mock payment workflow or Stripe integration. A mock gateway is preferred during early development because it allows the checkout and order-creation workflow to be tested without implementing real financial settlement.

4. Entity-Relationship Diagram (ERD)

The database is designed as a relational model containing the required entities:

Users

Products

Categories

Orders

Order_Items

Cart

Cart_Items

4.1 Entity Relationships

Relationship

Cardinality

Explanation

Users → Orders

1:N

One user can place many orders; each order belongs to one user.

Orders → Order_Items

1:N

One order contains one or more order items.

Products → Order_Items

1:N

One product can appear in many order items.

Categories → Products

1:N

One category can contain many products.

Users → Cart

1:1

Each user has one active cart in the MVP.

Cart → Cart_Items

1:N

One cart can contain multiple cart items.

Products → Cart_Items

1:N

One product can occur in cart items belonging to different carts.

The conceptual many-to-many relationships between Orders and Products and between Carts and Products are resolved through the associative entities Order_Items and Cart_Items.

4.2 Mermaid ERD

erDiagram

USERS ||--o{ ORDERS : places

USERS ||--|| CARTS : owns

ORDERS ||--|| ORDER_ITEMS : contains

PRODUCTS ||--o{ ORDER_ITEMS : ordered_in

CATEGORIES ||--o{ PRODUCTS : categorizes

CARTS ||--o{ CART_ITEMS : contains

PRODUCTS ||--o{ CART_ITEMS : added_to

USERS {

INTEGER id PK

VARCHAR email UK

VARCHAR password_hash

VARCHAR full_name

TIMESTAMP created_at

}

CATEGORIES {

INTEGER id PK

VARCHAR name UK

VARCHAR description

}

PRODUCTS {

INTEGER id PK

INTEGER category_id FK

VARCHAR name

VARCHAR description

```
    DECIMAL price

    INTEGER stock_quantity

    TIMESTAMP created_at
}
```

```
ORDERS {

    INTEGER id PK

    INTEGER user_id FK

    DECIMAL total_amount

    VARCHAR status

    TIMESTAMP created_at
}
```

```
ORDER_ITEMS {

    INTEGER id PK

    INTEGER order_id FK

    INTEGER product_id FK

    INTEGER quantity

    DECIMAL unit_price
}
```

```
CARTS {

    INTEGER id PK

    INTEGER user_id FK

    TIMESTAMP created_at

    TIMESTAMP updated_at
}
```

```
CART_ITEMS {

    INTEGER id PK

    INTEGER cart_id FK

    INTEGER product_id FK

    INTEGER quantity
}
```

4.3 Entity Attribute Specifications

USERS

Attribute

Data Type

Key / Constraint

id

INTEGER

PK

email

VARCHAR(255)

UNIQUE, NOT NULL

password_hash

VARCHAR(255)

NOT NULL

full_name

VARCHAR(100)

NOT NULL

created_at

TIMESTAMP

NOT NULL

CATEGORIES

Attribute

Data Type

Key / Constraint

id

INTEGER

PK

name

VARCHAR(100)

UNIQUE, NOT NULL

description

VARCHAR(500)

NULL allowed

PRODUCTS

Attribute
Data Type
Key / Constraint
id
INTEGER
PK
category_id
INTEGER
FK → CATEGORIES(id)
name
VARCHAR(200)
NOT NULL
description
VARCHAR(1000)
NULL allowed
price
DECIMAL(10,2)
NOT NULL
stock_quantity
INTEGER
NOT NULL
created_at
TIMESTAMP
NOT NULL
ORDERS

Attribute
Data Type
Key / Constraint
id
INTEGER
PK
user_id
INTEGER
FK → USERS(id)

total_amount

DECIMAL(10,2)

NOT NULL

status

VARCHAR(30)

NOT NULL

created_at

TIMESTAMP

NOT NULL

ORDER_ITEMS

Attribute

Data Type

Key / Constraint

id

INTEGER

PK

order_id

INTEGER

FK → ORDERS(id)

product_id

INTEGER

FK → PRODUCTS(id)

quantity

INTEGER

NOT NULL

unit_price

DECIMAL(10,2)

NOT NULL

CARTS

Attribute

Data Type

Key / Constraint

id

INTEGER

PK

user_id

INTEGER

FK → USERS(id), UNIQUE

created_at

TIMESTAMP

NOT NULL

updated_at

TIMESTAMP

NOT NULL

CART_ITEMS

Attribute

Data Type

Key / Constraint

id

INTEGER

PK

cart_id

INTEGER

FK → CARTS(id)

product_id

INTEGER

FK → PRODUCTS(id)

quantity

INTEGER

NOT NULL

5. Basic Database SQL Schema

The following SQL provides an implementation-oriented starting point for the ERD.

CREATE TABLE users (

id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

email VARCHAR(255) NOT NULL UNIQUE,

password_hash VARCHAR(255) NOT NULL,

full_name VARCHAR(100) NOT NULL,

created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP

```
);
```

```
CREATE TABLE categories (
```

```
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    name VARCHAR(100) NOT NULL UNIQUE,
```

```
    description VARCHAR(500)
```

```
);
```

```
CREATE TABLE products (
```

```
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    category_id INTEGER NOT NULL,
```

```
    name VARCHAR(200) NOT NULL,
```

```
    description VARCHAR(1000),
```

```
    price DECIMAL(10,2) NOT NULL CHECK (price >= 0),
```

```
    stock_quantity INTEGER NOT NULL CHECK (stock_quantity >= 0),
```

```
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
    CONSTRAINT fk_products_category
```

```
        FOREIGN KEY (category_id) REFERENCES categories(id)
```

```
);
```

```
CREATE TABLE orders (
```

```
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    user_id INTEGER NOT NULL,
```

```
    total_amount DECIMAL(10,2) NOT NULL CHECK (total_amount >= 0),
```

```
    status VARCHAR(30) NOT NULL,
```

```
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
    CONSTRAINT fk_orders_user
```

```
        FOREIGN KEY (user_id) REFERENCES users(id)
```

```
);
```

```
CREATE TABLE order_items (
```

```
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    order_id INTEGER NOT NULL,
```

```
    product_id INTEGER NOT NULL,
```

```

quantity INTEGER NOT NULL CHECK (quantity > 0),

unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),

CONSTRAINT fk_order_items_order

    FOREIGN KEY (order_id) REFERENCES orders(id),

CONSTRAINT fk_order_items_product

    FOREIGN KEY (product_id) REFERENCES products(id)

);

```

```

CREATE TABLE carts (

    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

    user_id INTEGER NOT NULL UNIQUE,

    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_carts_user

        FOREIGN KEY (user_id) REFERENCES users(id)

);

```

```

CREATE TABLE cart_items (

    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

    cart_id INTEGER NOT NULL,

    product_id INTEGER NOT NULL,

    quantity INTEGER NOT NULL CHECK (quantity > 0),

    CONSTRAINT fk_cart_items_cart

        FOREIGN KEY (cart_id) REFERENCES carts(id),

    CONSTRAINT fk_cart_items_product

        FOREIGN KEY (product_id) REFERENCES products(id),

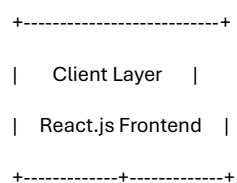
    CONSTRAINT uq_cart_product UNIQUE (cart_id, product_id)

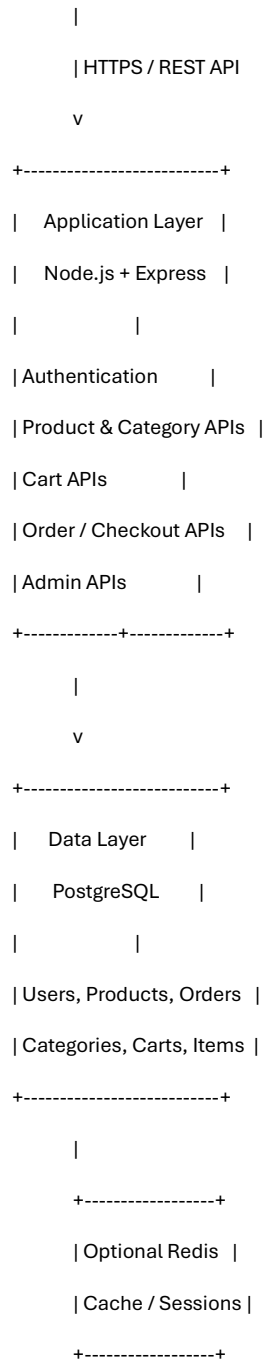
);

```

6. Architecture Overview

The proposed system follows a simple three-layer architecture:





7. Security Considerations

The initial architecture includes the following security practices:

Passwords will be stored using secure password hashing.

Authentication will use JWT-based access control.

HTTPS should be used for production communication.

Database queries should use parameterized queries/ORM mechanisms to reduce SQL injection risks.

Administrative APIs should require authorization.

Product stock and order totals must be validated on the server side.

Sensitive payment information should not be stored directly by the application when using a third-party payment gateway.

8. Sprint 1 Scope Summary

Sprint 1 establishes the foundation of the e-commerce project by defining the target audience, problem domain, MVP workflows, technology stack, database entities, relationships, and architectural structure.

The selected architecture provides a practical balance between simplicity, scalability, maintainability, data integrity, and semester-level feasibility. The ERD establishes clear primary-key and foreign-key relationships, while the MVP keeps the implementation focused on essential shopping operations.

9. Submission Checklist

Target Audience & Market Focus

Primary Persona

Core Pain Point

Domain Scope

MVP Feature Scope Matrix

Four to six primary workflows

Tech Stack Selection

Technical Justifications

ERD

Primary Keys and Foreign Keys

Cardinality and Relationships

SQL-compliant data types

Mermaid ERD

Basic SQL schema

Architecture overview

Security considerations

GitHub Submission

Create or use the dedicated project GitHub repository.

Create the file docs/SPRINT_1.md.

Paste/save this assignment as SPRINT_1.md.

Commit the document and associated assets to the primary branch.

Give the instructor and teaching assistant the required repository access.

Submit the repository URL through the LMS before the published deadline.